The Journal of Open Source Software

freegrad: alternative backward rules and gradient transforms alongside PyTorch autograd

Luigi Troiano [1], **Gioele Ciaparrone** [2], **and Genny Tortora** [1]

1 University of Salerno **2** Kebula

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 19 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Machine learning research often relies on backpropagation (LeCun et al., 2015; Rumelhart et al., 1986) as implemented in PyTorch's **autograd** (Paszke & others, 2019). While powerful, autograd enforces a strict symmetry between forward activations and their associated backward derivatives. This coupling can limit experimentation with **alternative gradient rules**, which are relevant for studying optimization dynamics, robustness, and theoretical properties of neural networks.

We present **freegrad**, a lightweight extension to PyTorch that allows researchers to define, register, and apply **custom backward rules** while leaving the forward pass unchanged. freegrad provides a registry of gradient transforms, a context manager for selective application, and wrappers for activation layers, enabling experiments on gradient clipping, stochastic perturbations (*gradient jamming*), or forward/backward decoupling.

Statement of need

Recent theoretical and empirical work has explicitly questioned the necessity of forward-backward symmetry in neural network training. Troiano et al. (Troiano et al., 2025) demonstrated that gradient direction, primarily determined by linear neuron connections, is the dominant factor in learning effectiveness, while exact gradient magnitudes derived from activation functions are largely redundant. They introduced constant, rectangular, triangular, and noisy gradients as alternatives, showing that neural networks—including CNNs and BNNs—can be trained effectively under these conditions. This motivates the need for tools that allow researchers to experiment with **custom backward rules** without patching PyTorch autograd.

freegrad directly addresses this need by providing a modular framework to replicate, extend, and generalize such experiments in a reproducible way.

Functionality

freegrad includes:

- **Registry of gradient rules**: built-in rules include `rectangular`, `triangular`, `clip_norm`, `noise`, and jamming variants (`full_jam`, `positive_jam`, `rectangular_jam`).
- **Context manager**: `with xg.use(rule, params, scope):` temporarily applies a rule during backpropagation.
- **Activation wrappers**: drop-in replacements for ReLU, Tanh, Sigmoid, etc., with custom backward functions controlled by the context.
- **Hooks**: optional parameter hooks for rule application at the parameter gradient level.
- **Functional API**: experimental support for custom `jvp` and `vjp`.

freegrad is tested with PyTorch >=2.0, documented with MkDocs, and distributed under the MIT license.

Custom Gradient Rules

freegrad allows users to define custom gradient transformations that modify the backward signal while keeping the forward computation unchanged. This section illustrates how to register new rules, compose existing ones, and—when needed—define fully custom backward passes using `torch.autograd.Function`.

Registering a Rule Function

The simplest way to introduce a new gradient rule is to define a Python function and register it using `xg.register`. A rule function must follow the signature:

[`fn(ctx, grad_out, tin, **params)` ,]

where:

- `ctx` is the `torch.autograd.Context` (rarely needed; may be `None`).
- `grad_out` is the incoming gradient ($\frac{\partial L}{\partial y}$).
- `tin` is the forward input tensor (x). When the rule is applied to parameter gradients (`scope="params"`), this value is `None`, and the rule must handle this case.
- `params` contains any optional parameters supplied through the context manager.

The rule must return the transformed gradient ($\frac{\partial L}{\partial x}$).

Example: Noisy Threshold Rule

The following rule applies a threshold to the input, masking out gradients for (x < t), and then adds Gaussian noise.

```python
import freegrad as xg
import torch

@xg.register("noisy_threshold")
def noisy_threshold(ctx, grad_out, tin, t: float = 0.0, sigma: float = 0.1):
    # Parameter-level gradients provide no input
    if tin is None:
        mask = 1.0
    else:
        mask = (tin >= t).to(grad_out.dtype)

    noise = sigma * torch.randn_like(grad_out)
    return grad_out * mask + noise
```

This rule can be activated during backpropagation using the `xg.use` context manager:

```python
import freegrad as xg
from freegrad.wrappers import Activation
import torch

x = torch.randn(5, requires_grad=True)
act = Activation("ReLU")

with xg.use("noisy_threshold", params={"t": 0.5, "sigma": 0.05}):
    y = act(x).sum()
    y.backward()
```

DRAFT

Composing Rules

freegrad also enables rule composition, allowing multiple transformations to be applied sequentially. The function `xg.compose(r1, r2, ...)` produces a new rule that applies the listed rules in order.

Example: Clipping Followed by Noise

```
import freegrad as xg
from freegrad.wrappers import Activation
import torch

# Compose clip_norm and noise into a single rule
clip_and_noise = xg.compose("clip_norm", "noise")

# Optional registration
xg.register("clip_then_noise")(clip_and_noise)

x = torch.randn(5, requires_grad=True)
act = Activation("Linear")

with xg.use(clip_and_noise, params={"max_norm": 1.0, "sigma": 0.1}, scope="activations")
    y = act(x).sum()
    y.backward()
```

Composition allows rapid prototyping of experimental gradient dynamics without modifying PyTorch internals.

Advanced Usage: Custom `torch.autograd.Function`

For scenarios requiring complete control over both forward and backward passes, a custom operator can be defined using the PyTorch `torch.autograd.Function` interface. Unlike rule functions, these custom autograd Functions have a **fixed backward pass** and do not interact with the `xg.use` mechanism.

Example: Square Function with Noisy Gradient

```
import torch

class SquareNoiseFn(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, sigma=0.1):
        ctx.save_for_backward(x)
        ctx.sigma = sigma
        return x**2

    @staticmethod
    def backward(ctx, grad_output):
        (x,) = ctx.saved_tensors
        noise = ctx.sigma * torch.randn_like(x)
        grad_in = grad_output * (2 * x) + noise
        return grad_in, None  # No gradient for sigma

def square_noise(x, sigma=0.1):
    return SquareNoiseFn.apply(x, sigma)
```

DRAFT

```python
# Example usage
x = torch.randn(5, requires_grad=True)
y = square_noise(x, sigma=0.1).sum()
y.backward()
print(x.grad)
```

This approach is appropriate when defining operations that cannot be expressed as standard activations or when precise backward customization is required.

Acknowledgements

We thank the open-source PyTorch community for providing the foundation on which freegrad builds.

References

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*, 436–444.

Paszke, A., & others. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*, 533–536.

Troiano, L., Gissi, F., Benedetto, V., & Tortora, G. (2025). Breaking the conventional forward-backward tie in neural networks: Activation functions. *Neurocomputing*, *654*, 131178. https://doi.org/10.1016/j.neucom.2025.131178

DRAFT